LangChain核心概念

- 来源: https://class.imooc.com/lesson/5282#mid=105027
- 阶段: 阶段四: 【框架应用】高效AI开发
- 周: 第8周 LangChain框架入门
- 模块: LangChain核心概念与应用
- 课节: 图文 1-2 LangChain核心概念

1-2 LangChain核心概念

LangChain核心概念

> [!TIP] 阅读指南 上一节我们通过一个具体案例感受了 LangChain 能解决哪些痛点。本节会从“俯视图”的角度，拆解 LangChain 的核心组件，说明每一块分别负责什么、它们如何协同工作，以及在什么场景下该重点关注哪一块。建议你一边阅读，一边对照自己正在做或准备做的 AI 应用，思考：这些组件中，哪些是你最想马上用起来的？

核心组件全景

LangChain 提供了 6 大核心组件:

1. Models - 大模型接口封装 2. Prompts - Prompt 模板管理 3. Chains - 串联多个步骤（简单线性流程；复杂编排见第7章 LangGraph） 4. Memory - 对话历史管理 5. Tools - 外部工具调用 6. Agents - 让 AI 自主决策

它们的关系就像做饭的工具:

- Models = 灶台(提供火力)
- Prompts = 菜谱(指导做什么)
- Chains = 流程(先炒菜再调味，但只能一条路走到底；复杂路线见第7章)
- Memory = 冰箱(记住之前做过什么)
- Tools = 厨具(切菜、炒锅等)
- Agents = 大厨(自己决定用什么工具)

LangChain 封装了各种大模型的 API,提供统一接口。

```
from langchain.chat_models import ChatOpenAI

# Chat 模型
chat_model = ChatOpenAI(
    model="qwen3.6-plus",
    temperature=0.7
)

# 调用
response = chat_model.invoke("原神好玩吗?")
```

好处:

- 统一接口 - 换模型只需改一行代码
- 参数标准化 - temperature、max_tokens 等统一管理
- 支持多种模型 - OpenAI、QWen、本地模型等

Prompts - 模板管理

不用每次手写 Prompt,用模板来管理。

```
from langchain.prompts import PromptTemplate

# 定义模板
template = """你是一个游戏攻略专家。

用户问题: {question}

请提供详细的攻略建议。
"""

prompt = PromptTemplate(
    template=template,
    input_variables=["question"]
)

# 使用
final_prompt = prompt.format(
    question="原神雷电将军怎么配队?"
)
```

- 复用 - 同一个模板用在多个场景
- 维护简单 - Prompt 集中管理
- 动态插值 - 自动填充变量

LangChain 提供了一种叫 LCEL 的管道语法，用 | 符号把多个处理步骤串联成一条流水线。

```
# 用 | 把 Prompt 和 Model 连起来
chain = prompt | chat_model

# 启动
result = chain.invoke({"question": "雷电将军怎么配队?"})
```

| 左边步骤的输出会自动成为右边步骤的输入，省去了手动传递中间变量的麻烦。但需要注意，Chain 本质是线性流水线——数据只能单向、顺序流动。

如果要用到更复杂的控制——比如条件分支、循环重试、多步骤协作——Chain 的线性模型就力不从心了。这时需要更强大的编排工具：LangGraph（下一章我们来讨论这个神器）。

Memory - 对话管理

AI 需要记住之前的对话，才能连贯交流。

```
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationChain

# 初始化 Memory
memory = ConversationBufferMemory()

# 创建对话 Chain
conversation = ConversationChain(
    llm=chat_model,
    memory=memory
)

# 多轮对话
response1 = conversation.predict(
    input="我在玩原神"
)
# AI: "原神是一款很受欢迎的开放世界游戏..."

response2 = conversation.predict(
    input="它有什么好玩的角色?"
)
# AI 知道"它"指的是原神,因为有 Memory
```

Memory 类型:

- ConversationBufferMemory - 保存全部对话
- ConversationSummaryMemory - 总结式记忆(节省 token)
- ConversationBufferWindowMemory - 只保留最近 N 轮

Tools - 工具调用

让 AI 能调用外部工具，比如查询 API、搜索等。

```
from langchain.tools import Tool

# 定义工具
def get_character_info(character_name):
    # 调用游戏 API 获取角色信息
    return {
        "name": character_name,
        "element": "雷",
        "weapon": "长柄武器"
    }

character_tool = Tool(
    name="查询角色信息",
    func=get_character_info,
    description="查询原神角色的基础信息,包括元素、武器类型等"
)

# 使用(后面会讲如何让 AI 自动调用)
result = character_tool.run("雷电将军")
```

Agents - 自主决策

前面我们已经认识了 Models、Prompts、Chains 和 Tools，这些都还可以看成是“普通函数”和“函数调用链”。从本节开始，会偶尔看到一个新名字：Agent（智能体）。在本章的这个位置，你不用立刻掌握它的所有细节，只需要先把它当成一个会自己决定用哪个工具的“总控函数”。

换句话说：当 Models、Prompts、Chains、Tools 都准备好了之后，Agent 就站在它们的最上面，负责回答三个问题：

- 现在需要调用哪些工具?
- 这些工具按什么顺序使用?
- 把工具的结果怎么组织成最终答案?

在现代 LangChain 中，我们使用 create_agent 来快速构建一个 Agent。

```
from langchain.agents import create_agent

# 准备工具列表
tools = [character_tool, team_tool, build_tool]

# 创建 Agent
agent = create_agent(
    llm=chat_model,
    tools=tools,
    system_prompt="你是一个资深游戏助手。"
)

# 使用 invoke 触发，传入消息序列
response = agent.invoke({
    "messages": [("user", "帮我分析一下雷电将军的最佳配队")]
})
print(response["messages"][-1].content)
```

Agent 的思考过程:

```
思考: 用户想了解雷电将军的配队和圣遗物
行动: 调用"查询角色信息"工具
观察: 获得雷电将军的基础信息

思考: 需要查询推荐配队
行动: 调用"推荐队伍"工具
观察: 获得配队建议

思考: 需要查询圣遗物搭配
行动: 调用"圣遗物推荐"工具
观察: 获得圣遗物建议

思考: 已有足够信息,可以回答
最终答案: [综合分析结果]
```

组件关系图

从简单到复杂

在实际项目里，你可以在不同层级使用 LangChain：

Level 1: 单次调用 (Model + Prompt)

```
chain = prompt | llm
result = chain.invoke({"input": "问题"})

1
2
```

Level 2: 多步流程 (Chain 串联)

```
overall_chain = chain1 | chain2 | chain3

1
```

需要工具调用或 Agent 功能时，LangChain 也提供了相应的组件，下一节我们会从完整应用的角度继续深入。

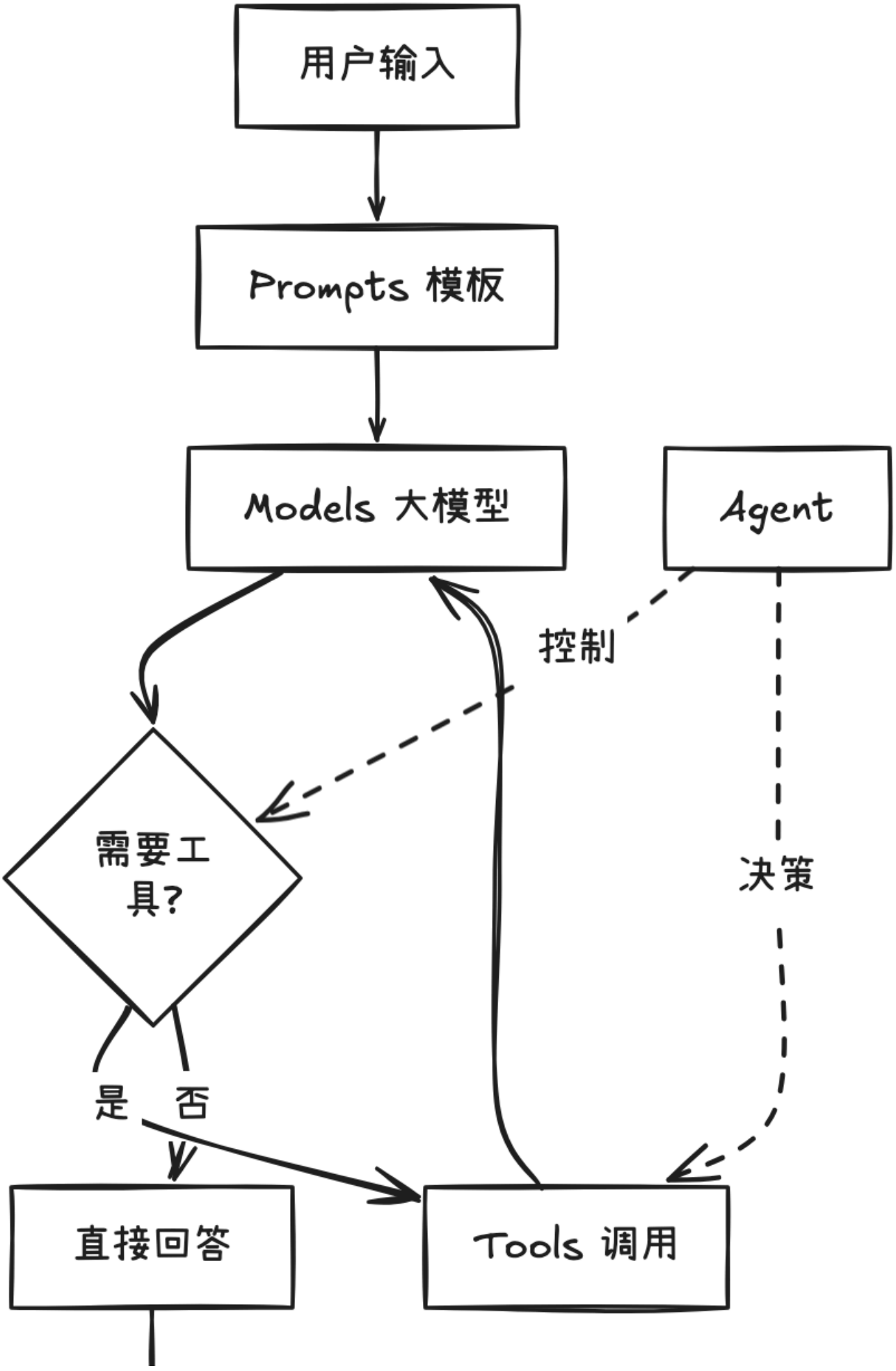
■ 学点英语

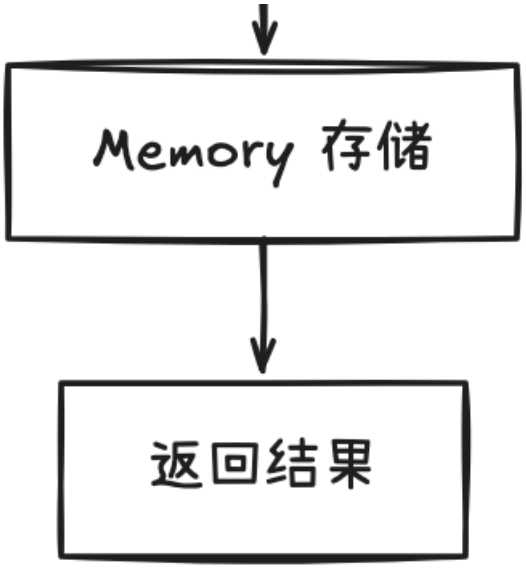
中文	English	音标	说明
链式表达式语言	LCEL (LangChain Expression Language)	/læŋ tʃeɪn ɪkˈspreʃn ˈlæŋgwɪdʒ/	LangChain 的管道语法，用 | 串联组件
流水线	Pipeline	/ˈpaɪplaɪn/	数据按顺序流经多个处理步骤的架构
可运行接口	Runnable Interface	/ˈrʌnəbl ˈɪntərfeɪs/	LangChain 组件的统一调用接口标准
智能体	Agent	/ˈeɪdʒənt/	能自主决策调用哪些工具的总控层

下一节预告

本节我们认识了 LangChain 的各种积木：Models、Prompts、Chains、Memory、Tools 和 Agents，但它们的真正力量要放到一个完整应用里才能看清楚。接下来，我们会用这些组件搭建一个原神攻略助手，从一次简单调用逐步发展到带记忆的对话系统。

图片


用户输入
Prompts 模板
Models 大模型
Agent
控制
需要工具?
决策
是
否
直接回答
Tools 调用


Memory 存储
返回结果